

Stage 10 FITS-IDI Visibility Recording Release Summary

Date: 2026-09-05

Repository branch: stage1-3-fx-engine

Target platform:

- Ubuntu 24.04.4
- GNU Radio 3.10.9.2
- Ettus B210 / UHD
- Python 3 with Astropy-based FITS writing

Purpose

Stage 10 adds optional recording of the experimentally validated Stage-9 coherent, fringe-stopped visibility stream to FITS-IDI, with a CSV sidecar for direct inspection and debugging.

The implemented design deliberately separates real-time GNU Radio DSP from file ownership:

- Stage 10A runs inside GNU Radio and publishes completed Stage-9 integrated visibility records over localhost TCP JSON lines.
- Stage 10B is a standalone logger application that owns operator Start/Stop control and writes FITS-IDI plus CSV output files.

The GNU Radio flowgraph does not create, open, close, or finalize FITS-IDI files. File writing starts only when the standalone logger is connected and the operator presses Start in the logger.

Final Architecture

The released Stage 10 architecture consists of:

- `grc/fx_interferometer_v1_stage10.grc`
 - Stage-9 flowgraph plus one Stage-10 visibility publisher and status sink.
 - Publishes the Stage-9 integrated stopped visibility.
 - Keeps high-rate DSP in existing GNU Radio blocks.
- `grc/fx_interferometer_v1_stage10_visibility_publisher.py`
 - GNU Radio publisher block.
 - Emits one JSON packet per completed Stage-9 integration.
 - Carries visibility values, source metadata, site metadata, baseline metadata, frequency metadata, integration metadata, and processing-state metadata.
 - Reports connection state to the GRC status display.
- `tools/stage10_protocol.py`
 - Shared packet schema and validation.
 - Defines schema version 2.
 - Computes retained FFT bins and effective correlated bandwidth from `samp_rate`, `fft_size`, and `visibility_edge_exclude_pct`.
- `tools/stage10_fitsidi_logger.py`
 - Standalone GUI logger.
 - Connects to the GNU Radio publisher.

- Displays live metadata and visibility samples.
 - Starts and stops FITS-IDI recording under explicit operator control.
 - Rejects invalid packets and stops recording if observation-defining metadata changes mid-file.
- `tools/stage10_fitsidi_writer.py`
 - Writes FITS-IDI files using Astropy only.
 - Writes diagnostic CSV sidecars.
 - Uses packet metadata rather than local defaults for science-critical values.
 - `tools/inspect_stage10_fitsidi.py`
 - Inspection utility for final or partial FITS-IDI files.
 - Prints source, array, UVW, sample-rate, FFT, edge-exclusion, retained-bin, and bandwidth metadata.
 - `tools/plot_stage10_fitsidi_uv.py`
 - Utility for plotting FITS-IDI U/V coverage.
 - `tools/stage10_packet_receiver.py`
 - Diagnostic receiver for inspecting raw Stage-10 publisher packets before file writing.

Major Implementation Steps

1. Stage 10 was reset to a clean Stage-9-derived flowgraph.

Earlier in-graph UVFITS/FITS attempts were removed. The final design starts from the validated Stage-9 flowgraph and adds only a low-rate publisher side path. This prevents file I/O and recording state from being embedded in the GNU Radio flowgraph.

2. The standalone logger became the sole owner of recording state.

The logger connects to the GRC publisher and can display live packets without creating files. FITS-IDI files are created only after the operator presses Start. Stop finalizes the partial file and renames it to a completed `.fitsidi` file.

3. Packet metadata was made explicit and mandatory.

Stage-10 packets now include source mode, source name, manual RA/Dec, site coordinates, baseline coordinates, sky centre frequency, sample rate, FFT size, visibility edge exclusion, retained FFT bins, effective correlated bandwidth, coherent integration time, `n_int`, fringe-stop state, delay-correction state, instrumental delay, polarization assumptions, emitted UTC, and integration-centre UTC.

4. Effective correlated bandwidth was corrected.

The bandwidth recorded in FITS-IDI and CSV is now the retained Stage-8 continuum bandwidth, not the raw B210 sample rate. It is calculated from integer FFT bins:

```
n_edge = int(fft_size * visibility_edge_exclude_pct / 100)
retained_fft_bins = fft_size - 2 * n_edge
effective_correlated_bandwidth_hz = retained_fft_bins * samp_rate / fft_size
```

For the current 30.72 MHz sample rate and 4096-point FFT:

```
5.0% edge exclusion -> 3688 retained bins -> 27.660000 MHz
12.5% edge exclusion -> 3072 retained bins -> 23.040000 MHz
20.0% edge exclusion -> 2458 retained bins -> 18.435000 MHz
```

5. Source metadata was corrected.

The publisher and logger now distinguish Sun and Manual RA/Dec mode. Manual observations use the source name `Manual` in live metadata, CSV rows, FITS source tables, and output filenames.

6. Recording control was moved out of GRC.

The previous GRC-side logging switch work was discarded. In the final architecture, GRC continuously publishes completed Stage-9 visibility packets when a logger is connected. The standalone logger Start/Stop buttons determine whether packets are written to file.

7. Metadata consistency checks were added.

The logger snapshots observation-defining metadata at Start. If metadata such as source, site, baseline, frequency, FFT size, edge exclusion, delay correction, fringe stopping, or polarization changes while recording, the logger finalizes the current file and enters an error state rather than mixing incompatible data in one FITS-IDI file.

8. The GRC embedded-block metadata problem was resolved.

GNU Radio 3.10.9.2 generated the Stage-10 publisher as:

```
self.stage10_visibility_publisher = stage10_visibility_publisher.blk()
```

This meant constructor metadata such as `visibility_edge_exclude_pct`, source mode, RA/Dec, sample rate, and FFT size were not passed into the publisher. The symptom was that the logger continued to show default metadata, especially 20% edge exclusion and 18.435 MHz, even when GRC showed 5%.

The final fix makes the publisher robust to that generated form. If GRC constructs the publisher with no metadata arguments, the publisher finds the generated top-block owner and reads the live GRC variables directly before each packet. This preserves correct live metadata even when the generated GRC line remains `blk()`.

FITS-IDI and CSV Output Behaviour

When recording is started in the standalone logger:

- A `.partial.fitsidi` file is created first.
- A matching diagnostic CSV sidecar is written.
- Records are appended only while logger state is RECORDING.
- Stop finalizes and verifies the FITS file.
- The partial FITS-IDI file is renamed to `.fitsidi`.
- The completed file does not grow after Stop.

The CSV sidecar includes packet-level fields for direct verification, including:

- Source mode and source name
- Manual RA and Dec
- Site latitude, longitude, and height
- Baseline E/N/U
- Sky centre frequency
- Sample rate
- FFT size
- Visibility edge exclusion
- Retained FFT bins
- Effective correlated bandwidth
- Effective integration time
- `n_int`
- Emitted UTC

- Integration-centre UTC
- Visibility real and imaginary values
- U/V/W coordinates

FITS-IDI Metadata Conventions

The FITS-IDI writer uses the following Stage-10 conventions:

- Baseline convention remains $B01 = RX1 - RX0$.
- Cross-spectrum convention remains $C01 = X0 * \text{conj}(X1)$.
- The recorded visibility is the measured Stage-9 integrated, fringe-stopped visibility.
- No conjugate duplicate baseline is written.
- $FLUX[0] = \text{real}(V_{\text{stopped}})$
- $FLUX[1] = \text{imag}(V_{\text{stopped}})$
- $FLUX[2] = 1.0$

FITS UVW values use:

```
UU = -project_u / c
VV = -project_v / c
WW = -project_w / c
```

Timing uses the integration-centre time:

```
integration_center_utc = emitted_utc - effective_integration_s / 2
```

The bandwidth metadata represents the retained correlated continuum bandwidth after edge exclusion, not the raw ADC/sample bandwidth.

Released Commits

The final Stage-10 release path is represented by these commits:

- c162e07 Reset Stage 10 FITS-IDI logging architecture
 - Rebuilt Stage 10 as a clean Stage-9-plus-publisher flowgraph and standalone logger architecture.
- 42281ac Clarify Stage 10 FITS bandwidth metadata
 - Clarified inspection and validation of retained-bandwidth metadata.
- f7fec0db Enforce Stage 10 metadata integrity
 - Added strict schema-versioned metadata handling, logger validation, metadata change detection, CSV/FITS metadata propagation, and end-to-end tests.
- d74d364 Fix Stage 10 GRC publisher metadata parameters
 - Added explicit publisher constructor metadata parameters to the Stage-10 GRC embedded wrapper.
- 1b64e26 Normalize Stage 10 publisher GRC metadata cache
 - Normalized the publisher GRC metadata cache to match the tuple form used by the working Stage 5 to Stage 9 embedded blocks.
- 4fd07d4 Read Stage 10 metadata from generated GRC owner
 - Final robust fix for GNU Radio 3.10.9.2 generating the publisher as `blk()`. The publisher now reads live metadata from the generated GRC top block before packet emission.

Earlier Stage-10 UVFITS and in-flowgraph recording attempts were superseded by the clean Stage-10 reset and are not part of the final released architecture.

Software Testing

The final Stage-10 test run covered publisher behaviour, packet protocol, FITS-IDI writing, and logger state handling:

```
python -m pytest \
  tests/test_stage10_publisher.py \
  tests/test_stage10_protocol.py \
  tests/test_stage10_fitsidi.py \
  tests/test_stage10_logger_state.py
```

Result:

20 passed, 24 warnings

The warnings were expected Astropy/FITS and IERS warnings in the test environment and did not indicate Stage-10 test failures.

Test coverage included:

- Stage-10 GRC remains Stage-9 plus publisher/status blocks only.
- Removed legacy Stage-10 in-graph recorder and UV logging controls are absent.
- Publisher packet source metadata changes between Sun and Manual modes.
- Publisher packet edge-exclusion metadata changes from live GRC values.
- Bare `stage10_visibility_publisher.blk()` generated-code case is handled correctly.
- Schema version 2 packets validate successfully.
- Missing or mismatched science metadata is rejected.
- Effective retained bandwidth is calculated from integer retained FFT bins.
- CSV and FITS metadata match packet metadata.
- Manual source metadata and filenames do not become Sun.
- Sun source metadata and filenames do not become Manual.
- File creation does not occur before logger Start.
- Stop finalizes partial FITS-IDI files.
- Metadata changes during recording trigger `ERROR_CONFIG_CHANGED`.
- FITS UVW sign and unit conventions are checked.

Hardware / Ubuntu Validation

Ubuntu/GRC validation confirmed that after the final publisher-owner metadata fix:

- The flowgraph can still generate the publisher line as `stage10_visibility_publisher.blk()`.
- The logger nevertheless receives live GRC metadata.
- Changing visibility edge exclusion in GRC to 5% produces:

```
Edge Exclusion: 5 %
Retained FFT Bins: 3688
Effective Correlated BW: 27.660000 MHz
```

- File logging starts and stops under standalone logger control.
- Source metadata and output filenames now match the selected GRC source mode.

Operational Procedure

1. Update the Ubuntu checkout:

```
cd ~/GNU_Radio/FX_GNU_radio_correlator
git pull
```

2. Start the standalone FITS-IDI logger:

```
cd ~/GNU_Radio/FX_GNU_radio_correlator
python3 tools/stage10_fitsidi_logger.py
```

3. Start GNU Radio Companion and run:

```
grc/fx_interferometer_v1_stage10.grc
```

4. Confirm the logger shows Publisher Connection = Connected.
5. Set GRC source and metadata before recording.
6. In the standalone logger, choose the observation name and output directory.
7. Press Start FITS-IDI Recording.
8. Press Stop FITS-IDI Recording to finalize the file.
9. Inspect the result:

```
python3 tools/inspect_stage10_fitsidi.py ~/FX_Correlator_Data/<file>.fitsidi
```
10. Optionally inspect the CSV sidecar directly to confirm per-record metadata.

Known Scope Limits

Stage 10 does not implement:

- Calibration solving
- Rho / correlation coefficient calibration
- Flux calibration
- Imaging
- Multi-B210 operation
- GNU Radio OOT module migration
- Real-time file writing inside the GRC flowgraph

Those remain future stages after Stage-10 experimental validation.